

Table of Contents

helpful sources.....	2
About NODE:.....	2
ES6 Evolution.....	3
first intro and installation.....	4
NPM – Node Package Manager.....	4
Install dependency.....	4
Install dev-dependencies.....	5
Versioning.....	5
MAJOR.....	6
MINOR.....	6
PATCHnvm.....	6
PREFIX explanation.....	6
Package.json vs package-lock-json.....	6
NodeJS.....	6
COMMONJS vs ESMModule.....	7
1. Syntax.....	7
2. File Extensions and Configuration.....	7
3. Synchronous vs Asynchronous Imports.....	7
4. Module Scope.....	7
5. Top-Level this and __dirname/ __filename.....	8
6. Interoperability.....	8
Summary:.....	8
Module export.....	8
import it to index.js.....	9
or function exports.....	9
main Core Modules we will work on:.....	10
Access FILENAME and DIRECTORY => CommonJS.....	10
Create folder in directory.....	11
Write to file.....	11
extend an file content with new input.....	12
Read file.....	12
Rename file.....	13
ESModule example.....	13
Access filename and director => ESMModule.....	13
Make directory and write file.....	14
OS Module.....	15
URL Module.....	16
HTTP.....	17
Nodemon activate.....	17
Add content type.....	18
create a public HTML file.....	19
Dynmaic filepath.....	19
initial content type.....	21
Error handling.....	22
Provide an API server.....	23
Render HTML.....	23
Render HTML from html filename in ESMModule.....	24
Event Listener.....	25
Initial Express.....	25
first message to screen.....	26

Send specific message like .sendStatus.....	26
Chain messages and JSON output.....	26
Download a file.....	27
handle file rendering.....	27
pass information from server to frontend.....	28
include css and other static files.....	29
Routing.....	30
Dynamic parameter in the url of USERS to get specifc user info.....	31
Reduce code if routes are similar.....	32
PARAMS method.....	33
Form input to save as json.....	34
EXPRESS JS.....	36
EXAMPLE with EJS, input field and public styling.....	36
Create Rest API with NodeJS and MongoDB.....	39
Setup API Route.....	40
Create Router for better structuring of the above API.....	41
Create CONTROLLER to manage node project propper.....	42
merge route actions which have the same route.....	43
Return body data in post request.....	43
Customize the error message which will come from express.error().....	45
Controller update to async handling.....	46
Install express async handler dependency.....	46

NodeJS Kursunterlagen

helpful sources

NPM packages.json introduction

<https://www.youtube.com/watch?v=e746Y8QBsQg>

About NODE:

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

As an asynchronous event-driven JavaScript runtime, Node.js is designed to build scalable network applications.

What is async?

In JavaScript, "**async**" refers to **asynchronous** operations, meaning that certain tasks can run in the background while the rest of the code continues to execute. This is useful when dealing with tasks that take time, like loading data from a server or reading files, so that the program doesn't "freeze" while waiting for them to complete.

Asynchronous programming allows JavaScript to handle tasks in the background while continuing to run other code.

- The key tools for async programming are **callbacks**, **promises**, and **async/await**.
- **Async** is crucial for improving the performance and responsiveness of web applications, especially when dealing with tasks that take time, like network requests.

ES6 Evolution

first intro
and

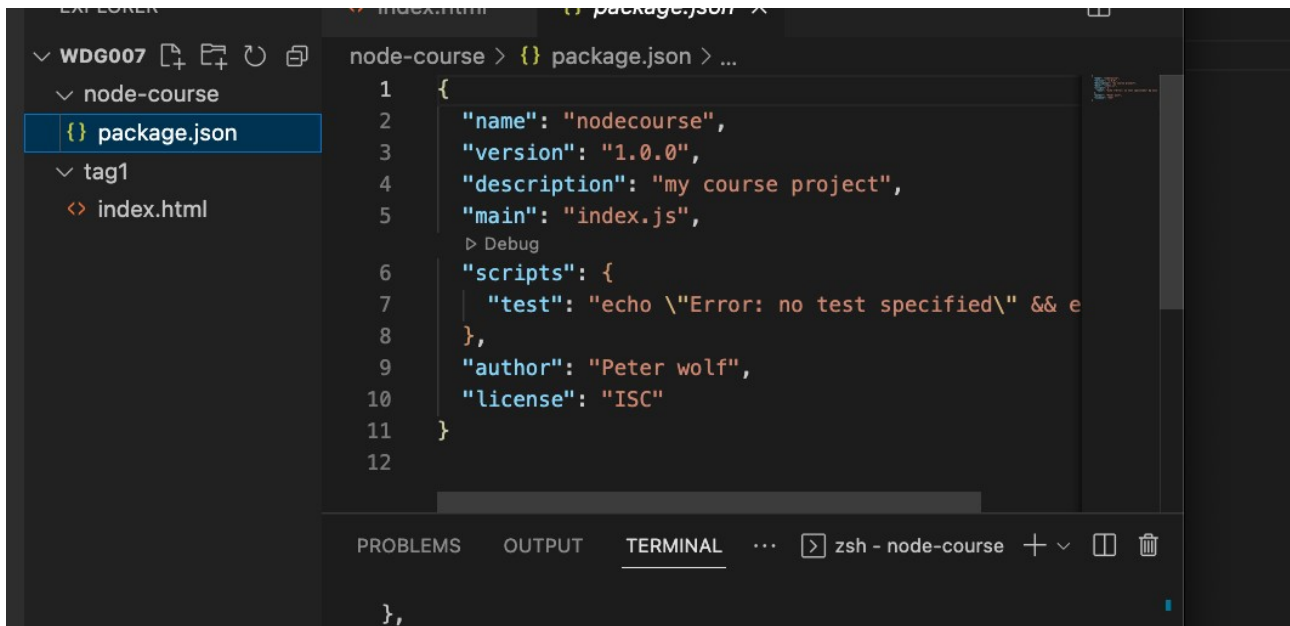


installation

Install node js with npm

Start with npm init in your folder to initialise your node enviroment (init generates a package.json)

the place where all dependencies will be stored



NPM – Node Package Manager

npm, Inc. is a company founded in 2014, and was acquired by GitHub in 2020. npm is a critical part of the JavaScript community and helps support one of the largest developer ecosystems in the world.

npm is lots of things.

- *npm is the package manager for [Node.js](#). It was created in 2009 as an [open source project](#) to help JavaScript developers easily share packaged modules of code.*
- *The npm Registry is a public collection of packages of open-source code for Node.js, [front-end web apps](#), [mobile apps](#), [robots](#), [routers](#), and countless other needs of the JavaScript community.*
- *npm is the command line client that allows developers to install and publish those packages.*

Node Package manager is a service of node which offers a huge amount of dependencies for node.

It comes automatically with installing node.js

Beside NPM there a different package manager like YARN.

Install dependency

Install UUID dependency

npm install uuid

npm install chalk (makes terminal output colored)

it will create a dependency in package.json

Install dev-dependencies

A dev dependency is only used in development mode. Will not installed in the production mode

Nodemon

```
npm install --save-dev nodemon //entweder
```

```
npm install -D nodemon //oder
```

```
npm install esLint
```

Delete modules folder to demo the power of package.json AND run npm install

Versioning

Semantik versioning

Incrementing semantic versions in published packages

To help developers who rely on your code, we recommend starting your package version at `1.0.0` and incrementing as follows:

Code status	Stage	Rule	Example version
First release	New product	Start with 1.0.0	1.0.0
Backward compatible bug fixes	Patch release	Increment the third digit	1.0.1
Backward compatible new features	Minor release	Increment the middle digit and reset last digit to zero	1.1.0
Changes that break backward compatibility	Major release	Increment the first digit and reset middle and last digits to zero	2.0.0

BEISPIEL: 5.0.1 => 5 (**MAJOR**) . 0 (**MINOR**) . 1 (**PATCH**)

MAJOR

If we update new features which are not compatible with the older version, then we have to change the MAJOR number

MINOR

we included a new Feature which is compatible with the version before => we change only the minor number

PATCH

if we updated something small (bugfix) which is compatible with the version before, then we update only the PATCH number

PREFIX explanation

TILDE ~

~ => install the major version with the minor version but always the latest patch

E.G ~5.0.3

this will install the 5th version with the 0 minor but if there is a latest version 5.0.6 available, it will install the latest patch.

ROOF ^

if you place the ^ before the version, you will always install the placed major version with the latest minor and patch .

STAR *

always the latest

ATTENTION: this is not recommended to use, because it can break your program

Package.json vs package-lock.json

<https://blog.bitsrc.io/package-vs-package-lock-json-file-d3b783dbaa95>

NodeJS

Source: <https://www.youtube.com/watch?v=fBNz5xF-Kx4>

<https://adrianmejia.com/getting-started-with-node-js-modules-require-exports-imports-npm-and-beyond/>

COMMONJS vs ESM module

Sources:

<https://blog.logrocket.com/commonjs-vs-es-modules-node-js/>

The key differences between **CommonJS** (CJS) and **ES Modules** (ESM) in Node.js relate to how modules are imported and exported. Here's a brief overview of their distinctions:

1. Syntax

- **CommonJS (CJS):** Uses `require()` to import modules and `module.exports` or `exports` to export them.

```
js
Copy code
// Import
const fs = require('fs');

// Export
module.exports = { myFunction };
```

- **ES Modules (ESM):** Uses `import` and `export` syntax, which is part of the JavaScript ES6+ standard.

```
js
Copy code
// Import
import fs from 'fs';

// Export
export const myFunction = () => {};
```

2. File Extensions and Configuration

- **CommonJS (CJS):** By default, Node.js treats `.js` files as CommonJS unless specified otherwise. No special configuration is needed.
- **ES Modules (ESM):** To use ESM in Node.js, you either:
 - Set `"type": "module"` in the `package.json` file.
 - Or use the `.mjs` file extension.

3. Synchronous vs Asynchronous Imports

- **CommonJS (CJS):** `require()` is synchronous, meaning the module is loaded and executed immediately when the `require()` call is encountered.
- **ES Modules (ESM):** `import` is asynchronous, allowing for optimizations like module loading in parallel or lazily (with `import()`).

4. Module Scope

- **CommonJS (CJS):** Exports are mutable and can be modified after being exported.
- **ES Modules (ESM):** Exports are immutable and have stricter bindings, ensuring that once something is exported, it cannot be changed by the importing file.

5. Top-Level `this` and `__dirname`/`__filename`

- **CommonJS (CJS):** `__dirname` and `__filename` are available to get the current directory and file path. Also, `this` refers to `module.exports`.
- **ES Modules (ESM):** `__dirname` and `__filename` are not available directly; they must be constructed using `import.meta.url`. `this` is undefined at the top level.

6. Interoperability

- **CommonJS (CJS)** can require ESM using dynamic `import()`, but importing CommonJS from ESM may involve additional steps due to differences in default exports.

Summary:

- **CJS:** Traditional Node.js module system (`require/module.exports`), synchronous, widely supported.
- **ESM:** New standard (`import/export`), asynchronous, follows JavaScript standards, better suited for modern development practices.

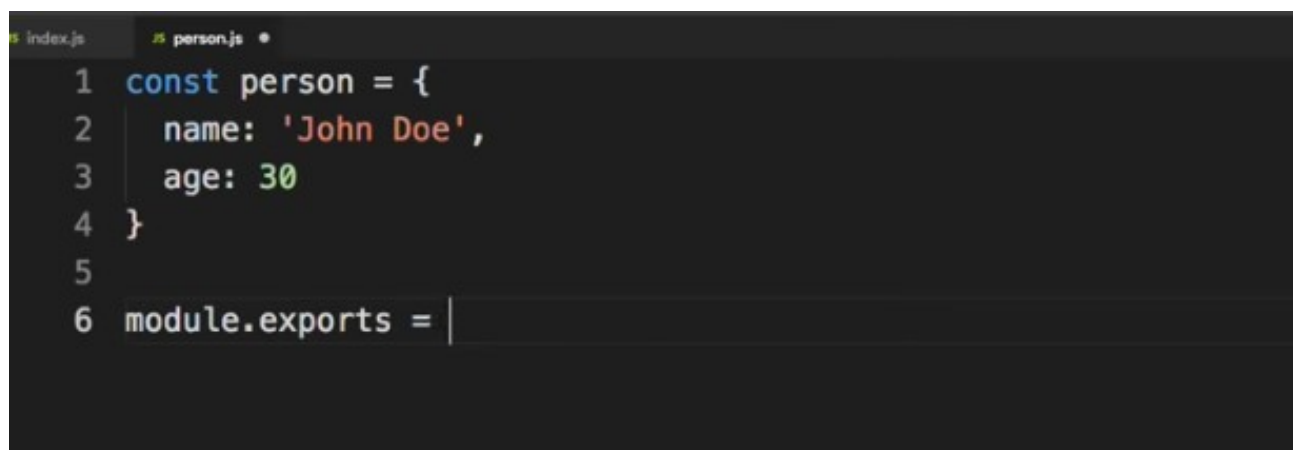
Module export

Export module to provide it to other modules

to make this happen we have to use `.mjs` instead of `.js` files

`module.exports = [modulename]` **WIRD DANN** `export default [modulename]`

!!TO BE UPDATED with **export default** in user!!



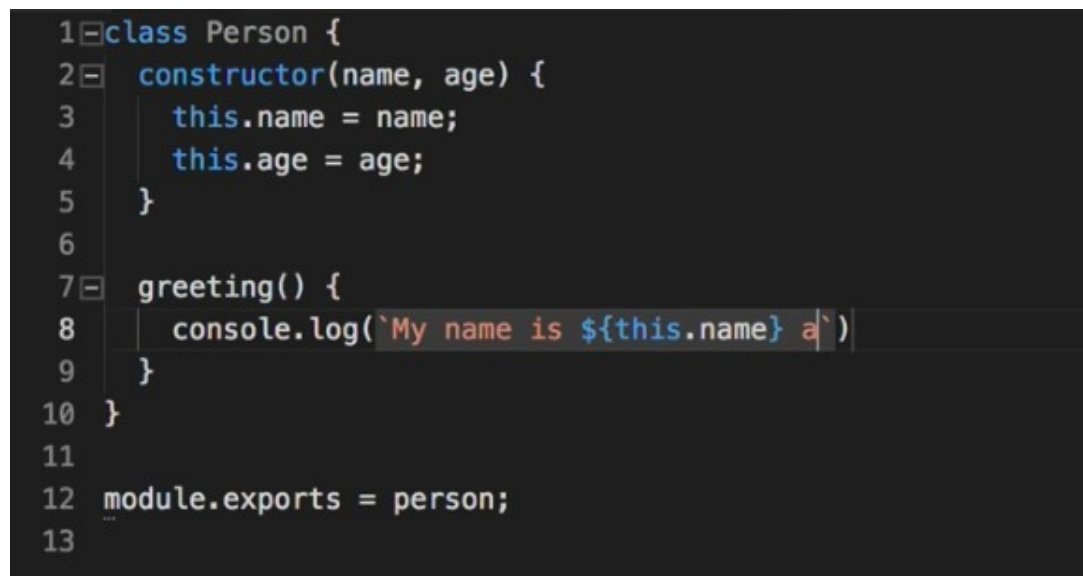
```
1 const person = {
2   name: 'John Doe',
3   age: 30
4 }
5
6 module.exports = |
```

import it to index.js

A screenshot of a code editor showing the contents of a file named 'index.js'. The code consists of three lines: a constant 'person' is assigned the result of 'require('./person)'; the next line is empty; and the third line is 'console.log(person.name);'. The file name 'index.js' is visible in the top left corner of the editor window.

```
1 const person = require('./person');
2
3 console.log(person.name);
4
```

or function exports

A screenshot of a code editor showing the contents of a file named 'person.js'. The code defines a 'Person' class with a constructor and a 'greeting' method. The constructor takes 'name' and 'age' as arguments and assigns them to 'this.name' and 'this.age'. The 'greeting' method logs a string that includes the person's name. Below the class definition, 'module.exports' is assigned the 'person' variable. Line numbers 1 through 13 are visible on the left side of the code.

```
1 class Person {
2   constructor(name, age) {
3     this.name = name;
4     this.age = age;
5   }
6
7   greeting() {
8     console.log(`My name is ${this.name} a`);
9   }
10 }
11
12 module.exports = person;
13
```

Const [filename] = require([filepath]) **WIRD ZU** import [file] from [filename]

```
1 const Person = require('./person');  
2  
3 const person1 = new Person('John Doe', 30);  
4  
5 person1.greeting();  
6
```

main Core Modules we will work on:

PATH

Import path = require('path');

FS

import fs = require('fs');

Access FILENAME and DIRECTORY => CommonJS

__filename (gives actual file name)

__dirname (gives actual directory name)

console.log(__dirname, __filename)

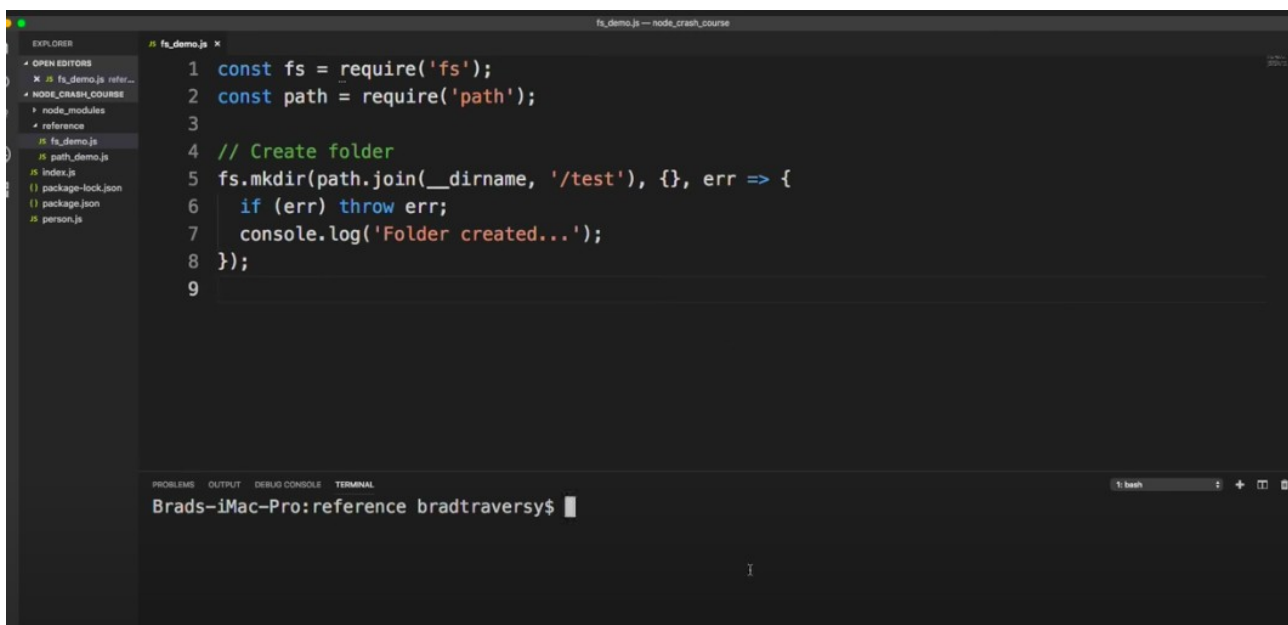
```
4 console.log(path.basename(__filename));
5
6 // Directory name
7 console.log(path.dirname(__filename));
8
9 // File extension
10 console.log(path.extname(__filename));
11
12 // Create path object
13 console.log(path.parse(__filename).base);
14
15 // Concatenate paths
16 console.log(path.join(__dirname, 'test', 'hello.html'));
17
```

Create folder in directory



```
1 const fs = require('fs');
2 const path = require('path');
3
4 // Create folder
5 fs.mkdir(path.join(__dirname, '/test'), {}, err => {
6   if (err) throw err;
7   console.log('Folder created...');
8 });
9
```

Write to file

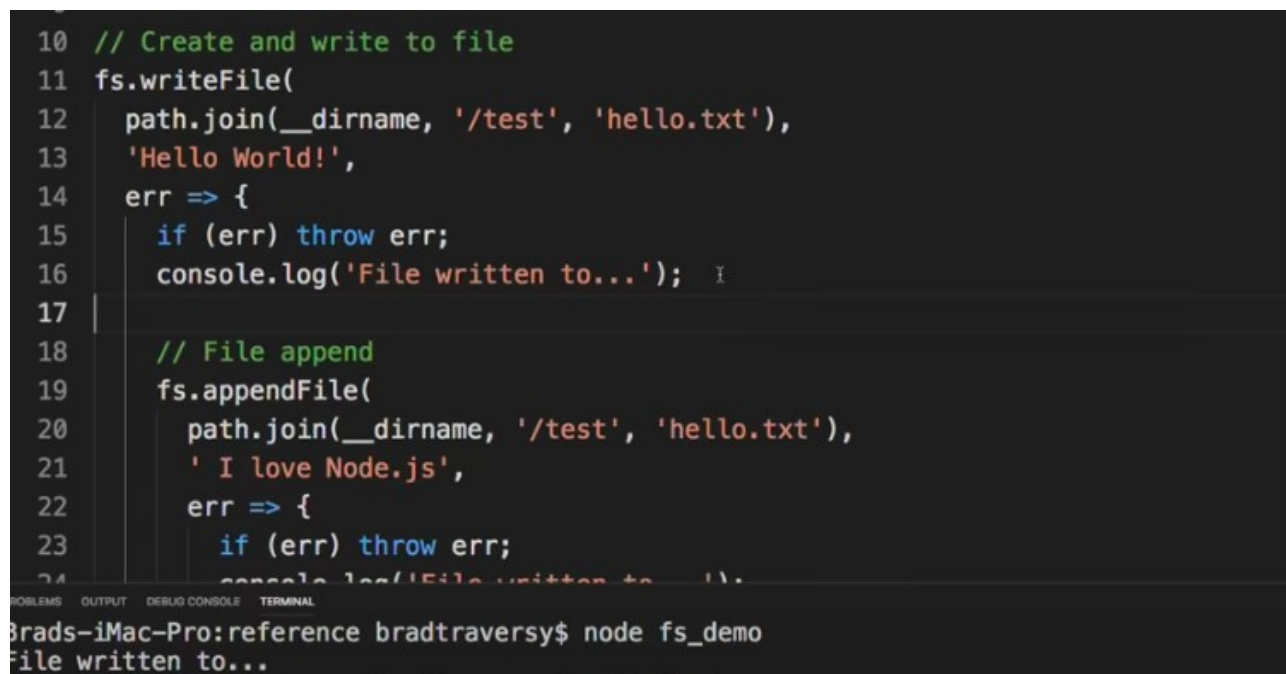


```
1 const fs = require('fs');
2 const path = require('path');
3
4 // Create folder
5 fs.mkdir(path.join(__dirname, '/test'), {}, err => {
6   if (err) throw err;
7   console.log('Folder created...');
8 });
9
```

Brads-iMac-Pro:reference bradtraversy\$

extend an file content with new input

fs.appendFile I can attach content to an existing filename



```
10 // Create and write to file
11 fs.writeFile(
12   path.join(__dirname, '/test', 'hello.txt'),
13   'Hello World!',
14   err => {
15     if (err) throw err;
16     console.log('File written to...');
17   }
18
19 // File append
20 fs.appendFile(
21   path.join(__dirname, '/test', 'hello.txt'),
22   ' I love Node.js',
23   err => {
24     if (err) throw err;
25     console.log('File written to...');
26   }
27 )
```

Brads-iMac-Pro:reference bradtraversy\$ node fs_demo

File written to...

appendFile has to be placed in the callback (error) function of writeFile to make it work

Read file

UTF8 has to be second parameter

```

29
30 // Read file
31 fs.readFile(path.join(__dirname, '/test', 'hello.txt'), 'utf8', (err, data) => {
32   if (err) throw err;
33   console.log(data);
34 });
35

```

Rename file

```

36 // Rename file
37 fs.rename(
38   path.join(__dirname, '/test', 'hello.txt'),
39   path.join(__dirname, '/test', 'helloworld.txt'),
40   err => {
41     if (err) throw err;
42     console.log('File renamed...');
43   }
44 );
45

```

First parameter is origin and second is new

ESModule example

Access filename and director => ESMODULE

In Node.js using ECMAScript Modules (ESM), obtaining the current directory (`__dirname` in CommonJS) is slightly different because `__dirname` and `__filename` are not directly available in ESM by default.

To get the equivalent of `__dirname` in an ESM environment, you can construct it using the `import.meta.url` property in combination with the `path` module.

```

JS minindex.js > ...
1  import { dirname } from 'path';
2  import { fileURLToPath } from 'url';
3
4  // Get the current file URL and convert it to a path
5  const __filename = fileURLToPath(import.meta.url);
6  const __dirname = dirname(__filename);
7
8  console.log('Directory name:', __dirname);

```

Make directory and write file

```
JS mindex.js > ...
1 import { dirname, join } from 'path';
2 import { fileURLToPath } from 'url';
3 import * as fs from 'fs'
4
5 // Get the current file URL and convert it to a path
6 const __filename = fileURLToPath(import.meta.url);
7 const __dirname = dirname(__filename);
8
9 console.log('Directory name:', __dirname);
10
11
12 fs.mkdir(join(__dirname, '/testMfile'), {}, err => {
13   if (err) throw err;
14   console.log("folder created")
15 })
16 fs.writeFile(join(__dirname, '/testMfile', 'test.txt'), 'Hello peter', err => {
17   if (err) throw err;
18   console.log("file written to project");
19
20   fs.appendFile(join(__dirname, '/testMfile', 'test.txt'), 'I love coding', err => {
21     if (err) throw err;
22     console.log("append text")
23   })
24 })
25 })
```

OS Module

Operating system

with ESM module the import is **‘import os from ‘os’**”

```
os_demo.js x
1  const os = require('os');
2
3  // Platform
4  console.log(os.platform());
5
6  // CPU Arch
7  console.log(os.arch());
8
9  // CPU Core Info
10 console.log(os.cpus());
11
12 // Free memory
13 console.log(os.freemem());
14
15 // Total memory
16 console.log(os.totalmem());
17
18 // Home dir
19 console.log(os.homedir());
20
21 // Uptime
22 console.log(os.uptime());
23
```

URL Module

The screenshot shows the VS Code editor with a file named `url_demo.js` open. The code defines a `URL` object and logs its properties. The terminal shows the output of the script.

```
1 const url = require('url');
2
3 const myUrl = new URL(
4   'http://mywebsite.com/hello.html?id=100&status=active'
5 );
6
7 // Serialized URL
8 console.log(myUrl.href);
9 console.log(myUrl.toString());
10 // Host (root domain)
11 console.log(myUrl.host);
12 // Hostname (does not get pq)
13 console.log(myUrl.hostname);
14
```

Terminal Output:

```
http://mywebsite.com/hello.html?id=100&status=active
mywebsite.com
Brads-iMac-Pro:reference bradtraversy$ node url_demo
http://mywebsite.com/hello.html?id=100&status=active
http://mywebsite.com/hello.html?id=100&status=active
mywebsite.com
mywebsite.com
Brads-iMac-Pro:reference bradtraversy$ node url_demo
http://mywebsite.com:8000/hello.html?id=100&status=active
http://mywebsite.com:8000/hello.html?id=100&status=active
mywebsite.com:8000
mywebsite.com
Brads-iMac-Pro:reference bradtraversy$
```

The screenshot shows the VS Code editor with the `url_demo.js` file. The code now includes logic to append a new parameter and loop through the search parameters. A tooltip is visible over the `log` function. The terminal shows the updated output.

```
11 console.log(myUrl.hostname);
12 // Pathname
13 console.log(myUrl.pathname);
14 // Serialized query
15 console.log(myUrl.search);
16 // Params object
17 console.log(myUrl.searchParams);
18 // Add param
19 myUrl.searchParams.append('abc', '123');
20 console.log(myUrl.searchParams);
21 // Loop through params
22 myUrl.searchParams.forEach((value, name) => console.log(`${name}: ${value}`));
23
```

Tooltip:

```
log(message?: any,
...optionalParams: any[]):
void
Prints to stdout with newline.
```

Terminal Output:

```
Brads-iMac-Pro:reference bradtraversy$ node url_demo
http://mywebsite.com/hello.html?id=100&status=active
http://mywebsite.com/hello.html?id=100&status=active
mywebsite.com
mywebsite.com
/hello.html
?id=100&status=active
URLSearchParams { 'id' => '100', 'status' => 'active' }
URLSearchParams { 'id' => '100', 'status' => 'active', 'abc' => '123' }
Brads-iMac-Pro:reference bradtraversy$ node url_demo
```

HTTP

Basic set up

```
1  const http = require('http');
2
3  // Create server object
4  http
5    .createServer((req, res) => {
6      // Write response
7      res.write('Hello World');
8      res.end();
9    })
10   .listen(5000, () => console.log('Server running...'));
11
```

with statement

Nodemon activate

Include nodemon in the scripts area in the package.json

run the file then with NPM RUN <scriptname>

```
1  {
2    "name": "node_crash_course",
3    "version": "1.0.0",
4    "description": "Node crash course",
5    "main": "index.js",
6    "scripts": {
7      "start": "node index",
8      "dev": "nodemon index"
9    },
10   "author": "Brad Traversy",
11   "license": "ISC",
12   "dependencies": {
13     "uuid": "^3.3.2"
14   },
15 }
```

Add content type

with `res.writeHead` we can update the content type to get a proper html content type

```
index.js  x  11 package.json
2  const path = require('path');
3  const fs = require('fs');
4
5  const server = http.createServer((req, res) => {
6    if (req.url === '/') {
7      res.writeHead(200, { 'Content-Type': 'text/html' });
8      res.end('<h1>Home</h1>');
9    }
10  });
11
12  const PORT = process.env.PORT || 5000;
13
14  server.listen(PORT, () => console.log(`Server running on port $
    {PORT}`));
```

Routing exercise Routing

<https://www.geeksforgeeks.org/routing-in-node-js/>

Now create a server by adding the following lines of code:

```
http.createServer(function (req, res) {
  res.write('Hello World!'); // Write a response
  res.end(); // End the response
}).listen(3000, function() {
  console.log("server start at port 3000"); // The server object listens on port 3000
});
```

Now add the following lines of code in the above function to perform routing:

```
const server = http.createServer((req, res) => {
  const url = req.url
  if (url === "/") {
    res.write("Homepage");
    res.end();
  } else if (url === "/aboutme") {
    res.write("about me");
    res.end();
  } else if (url === "/users") {
    res.end(JSON.stringify(responseData));
  } else {
    res.write("hello world");
    res.end();
  }
})
```

create a public HTML file

we will import the html file to the server page

```
1  const http = require('http');
2  const path = require('path');
3  const fs = require('fs');
4
5  const server = http.createServer((req, res) => {
6    if (req.url === '/') {
7      fs.readFile(
8        path.join(__dirname, 'public', 'index.html'),
9        (err, content) => {
10         if (err) throw err;
11         res.writeHead(200, { 'Content-Type': 'text/html' });
12         res.end(content);
13       }
14     }
15   });
```

if we need a new path we need to create an second if() statement

with several levels of page paths it will be not efficient to provide every time an if statement

we will recreate the if() checks

additional to that we want to provide a constant HTML filename

Dynmaic filepath

We will create a dynamic filepath to provide the actual access path dynamically

it will check in the console.log the acutall dynamic path

```
5
6  // Build file path
7  let filePath = path.join(
8    __dirname,
9    'public',
10    req.url === '/' ? 'index.html' : req.url
11  );
12
13  console.log(filePath);
14  res.end();
15  });
```

initial content type

Therefore we need to have the extensions of the file (path.ext())

1. extract the extension of the variable filePath (image above) and provide it in an new variable.
2. we will provide a default contentType variable with “text/html”;
3. check with an switch statement the extension and change the contentType.

```
// Extension of file
let extname = path.extname(filePath);

// Initial content type
let contentType = 'text/html';
```

```
// Check ext and set content type
switch (extname) {
  case '.js':
    contentType = 'text/javascript';
    break;
  case '.css':
    contentType = 'text/css';
    break;
  case '.json':
    contentType = 'application/json';
    break;
  case '.png':
    contentType = 'image/png';
    break;
```

Error handling

We will provide several error levels with a final success

1. read the filename
2. check if err at all
3. in the error checking we will check for the `err.code === ENOENT` (no file found error) => we will then provide a page not found page (error.html)
4. else other all other error we will provide a serverHead 500 and send error message to frontpage
5. after the if error check we set it to `else{ successful page providing}`

```
8 // Read File
9 fs.readFile(filePath, (err, content) => {
10   if(err) {
11     if(err.code === 'ENOENT') {
12       // Page not found
13       fs.readFile(path.join(__dirname, 'public', '404.html'), (err, content)
14         => {
15         res.writeHead(200, { 'Content-Type': 'text/html' });
16         res.end(content, 'utf8');
17       })
18     } else {
19       // Some server error
20       res.writeHead(500);
21       res.end(`Server Error: ${err.code}`);
22     }
23   } else {
24     // Success
25     res.writeHead(200, { 'Content-Type': contentType });
26     res.end(content, 'utf8');
```

Provide an API server

We will create our own api path to provide a json formatted output

```
13     }
14   );
15 }
16
17 if (req.url === '/api/users') {
18   const users = [
19     { name: 'Bob Smith', age: 40 },
20     { name: 'John Doe', age: 30 }
21   ];
22   res.writeHead(200, { 'Content-Type': 'application/json' });
23   res.end(JSON.stringify(users));
24 }
25 });
26
```

Render HTML

```
src > JS server.js > http.createServer() callback
1  import http from 'http'
2
3  http.createServer((req, res)=>{
4
5    if(req.url === "/users"){
6
7      res.writeHead(200, { 'Content-Type': 'text/html' });
8
9      // Write the HTML content to the response body
10     res.write(`
11     <!DOCTYPE html>
12     <html lang="en">
13     <head>
14       <meta charset="UTF-8">
15       <meta name="viewport" content="width=device-width, initial-scale=1.0">
16       <title>Node.js HTML Response</title>
17     </head>
18     <body>
19       <h1>Hello from Node.js!</h1>
20       <p>This is a simple HTML page rendered by Node.js HTTP module.</p>
21     </body>
22     </html>
23   `);
24
25   // End the response
26   res.end();
27 }
28 })
29
30 .listen(3333, ()=>console.log("server runs on 3333"))
```


Render HTML from html filename in ESM module

```
src > JS server.js > [E] server > [E] http.createServer() callback > [E] html > [E] fs.readFile('utf-8') callback
1 import http from 'http'
2 import * as fs from 'fs'
3 import { dirname, join } from 'path';
4 import { fileURLToPath } from 'url';
5
6 const __filename = fileURLToPath(import.meta.url);
7 const __dirname = dirname(__filename)
8 const server = http.createServer(async (req, res) => {
9   try {
10     // Read the HTML file asynchronously
11     const html = await fs.readFile(join(__dirname, './html/index.html'), 'utf-8',
12       (err, data) => {
13
14         if (err) throw err;
15         // Set the response header to indicate the content type (HTML)
16         res.writeHead(200, { 'Content-Type': 'text/html' });
17
18         // Send the HTML content to the client
19         res.end(data);
20       });
21   } catch (error) {
22     // Handle errors (e.g., file not found)
23     res.writeHead(500, { 'Content-Type': 'text/plain' });
24     res.end('Error loading the HTML file.');
```

```
25 console.error('Error reading the HTML file:', error);
26 }
27 })
28 .listen(3333, () => console.log("server runs on 3333"))
```


Event Listener

https://www.youtube.com/watch?v=XF1_MlZ5l6M&list=WL&index=3

Routing with NodeJS – HTTP module.exports

<https://www.geeksforgeeks.org/routing-in-node-js/>

EXPRESS JS

Now there is a more elegant way to handle routing.
Express JS makes our life more easy.

Initial Express

```
const express = require("express")
const app = express()

app.get('/', (req, res) => {

})

app.listen(3000)
```

Express has all possible methods available
the main used ones are

app.get
app.post
app.delete
app.put

first message to screen

```
const express = require("express")
const app = express()

app.get("/", (req, res) => {
  console.log("Here")
  res.send("Hi")
})

app.listen(3000)
```

.send is not really used. It is too unspecific

Send specific message like .sendStatus

```
JS server.js
const express = require("express")
const app = express()

app.get("/", (req, res) => {
  console.log("Here")
  res.sendStatus(500)
})

app.listen(3000)
```

Chain messages and JSON output

```
JS server.js
const express = require("express")
const app = express()

app.get("/", (req, res) => {
  console.log("Here")
  res.status(500).json({ message: "Error" })
})

app.listen(3000)
```

Download a file

create a file which can be downloaded

```
JS server.js
const express = require("express")
const app = express()

app.get("/", (req, res) => {
  console.log("Here")
  res.download("server.js")
})

app.listen(3000)
```

handle file rendering

all files for the VIEW will be normally stored in folder views
so create the folder and put files in there.

```
JS server.js
const express = require("express")
const app = express()

app.get("/", (req, res) => {
  console.log("Here")
  res.render("index")
})

app.listen(3000)
```

ERROR APPEARS

if we open the page we will see an error.
with this we cannot provide a html file yet.

We have to install a view engine like PUG or EJS to read the files properly.

Then we have to initialise EJS with `app.set("view engine", "ejs")` and rename the file from `.html` to `.ejs`

include the extension `ejs` to VSC to see highlights

```
JS server.js
const express = require("express")
const app = express()

app.set('view engine', 'ejs')

app.get("/", (req, res) => {
  console.log("Here")
  res.render("index")
})
```

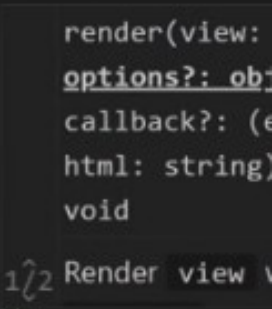
pass information from server to frontend

with `ejs` we can provide variables to the frontend

```
JS server.js
const express = require("express")
const app = express()

app.set("view engine", "ejs")

app.get("/", (req, res) => {
  console.log("Here")
  res.render("index", { text: "World" })
})
```



after the filename we can provide a second parameter

in the frontend we can access the data with `<%= content %>`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-sca
  <title>Document</title>
</head>
<body>
  Hello <%= text %>
</body>
</html>
```

Hint! If the TEXT variable is not defined, an error will be appear.
To avoid that error we can use locals. - object.
Locals. -object will always defined.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-sca
  <title>Document</title>
</head>
<body>
  Hello <%= locals.text || 'Default' %>
</body>
</html>
```

Quelle für andere optionen zum anwenden des ESJ

<https://www.digitalocean.com/community/tutorials/how-to-use-ejs-to-template-your-node-index.application>

include css and other static files

You have to use the .use method to implement the static files

```
index.js > ...
const express = require('express');
const app = express();

app.use(express.static(__dirname + '/views'));

/**
 * gängigste methoden für express app
 *
 * app.get();
 * app.post();
```

Routing

Different routes providing

```
const express = require("express")
const app = express()

app.set("view engine", "ejs")

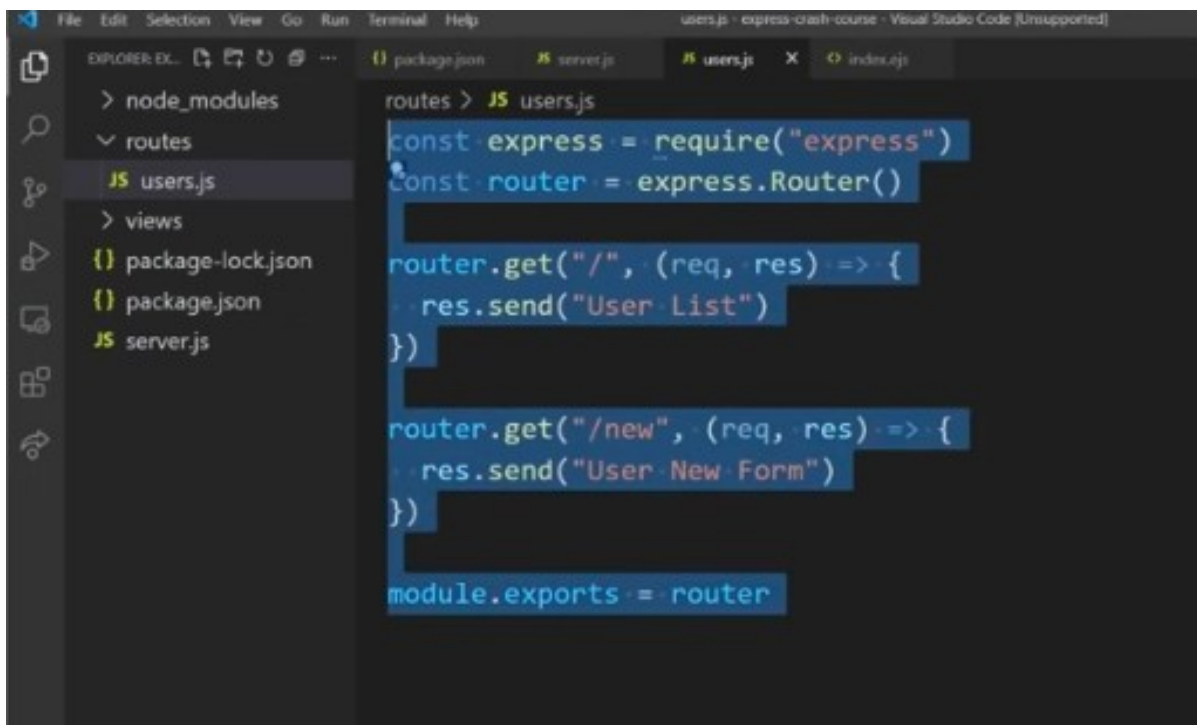
app.get("/", (req, res) => {
  console.log("Here")
  res.render("index", { text: "World" })
})

app.get("/users", (req, res) => {
  res.send("User List")
})

app.get("/users/new", (req, res) => {
  res.send("User New Form")
})

app.listen(3000)
```

export routes in external router file



1. create new folder called router (name is not fixed, but conventions are router)
2. create a file for all routes you use for e.g users
3. initialize an express app and call express.Router() to start a SMALL SERVER
4. you can now delete all users parts of the routes if all routes in the file are heading to .../users/....
5. export module
6. import module to index.js

```
//render page als reqs datei
//alternativ kan ich dateien vom server in das fronte
res.render("homepage",{text: "hier ist mein text aus

});

app.get('/downloads', (req, res)=>{
  res.download("public/test.txt")
})

//import router and instetiate the routes
const userRouter = require('./routes/users.cjs');
app.use('/users', userRouter)

app.listen(3500, ()=>console.log("port 3500"));
```

Dynamic parameter in the url of USERS to get specfic user info

With params you can identify spezific user information dynamically

the param can be named as whatever you want like (:id, :dog)

access to the id is with req.params.<name of param>

```
router.post("/", (req, res) => {
  res.send("Create User")
})

router.get("/:id", (req, res) => {
  res.send(`Get User With ID ${req.params.id}`)
})

module.exports = router
```

HINT: the routes run from top to bottom...so if you use dynamic routes, and one of them match, it will run...

Example:

we have this right setup

change the url to

we want to create a new user so

run the route `users/new...`

`localhost:PORT/users/new`

it happen, that the output is not the form

it is the message "Get user with id new"

```
routes > JS users.js
const express = require("express")
const router = express.Router()

router.get("/", (req, res) => {
  res.send("User List")
})

router.post("/", (req, res) => {
  res.send("Create User")
})

router.get("/:id", (req, res) => {
  res.send(`Get User With ID ${req.params.id}`)
})

router.get("/new", (req, res) => {
  res.send("User New Form")
})

module.exports = router

TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE
[nodemon] restarting due to changes...
```

Reduce code if routes are similar

We have initiate several routes in `users.cjs`

example right:

we have

`.get`

`.post`

`.put`

`.delete`

we can combine them in the `users.cjs`

like....

```
routes > JS users.js
})

router.post("/", (req, res) => {
  res.send("Create User")
})

router.get("/:id", (req, res) => {
  res.send(`Get User With ID ${req.params.id}`)
})

router.put("/:id", (req, res) => {
  res.send(`Update User With ID ${req.params.id}`)
})

router.delete("/:id", (req, res) => {
  res.send(`Delete User With ID ${req.params.id}`)
})
```



```

router
  .route("/:id")
  .get((req, res) => {
    res.send(`Get User With ID ${req.params.id}`)
  })
  .put((req, res) => {
    res.send(`Update User With ID ${req.params.id}`)
  })
  .delete((req, res) => {
    res.send(`Delete User With ID ${req.params.id}`)
  })

module.exports = router

```

PARAMS method

```

routes > JS users.js
  .get((req, res) => {
    res.send(`Get User With ID ${req.params.id}`)
  })
  .put((req, res) => {
    res.send(`Update User With ID ${req.params.id}`)
  })
  .delete((req, res) => {
    res.send(`Delete User With ID ${req.params.id}`)
  })

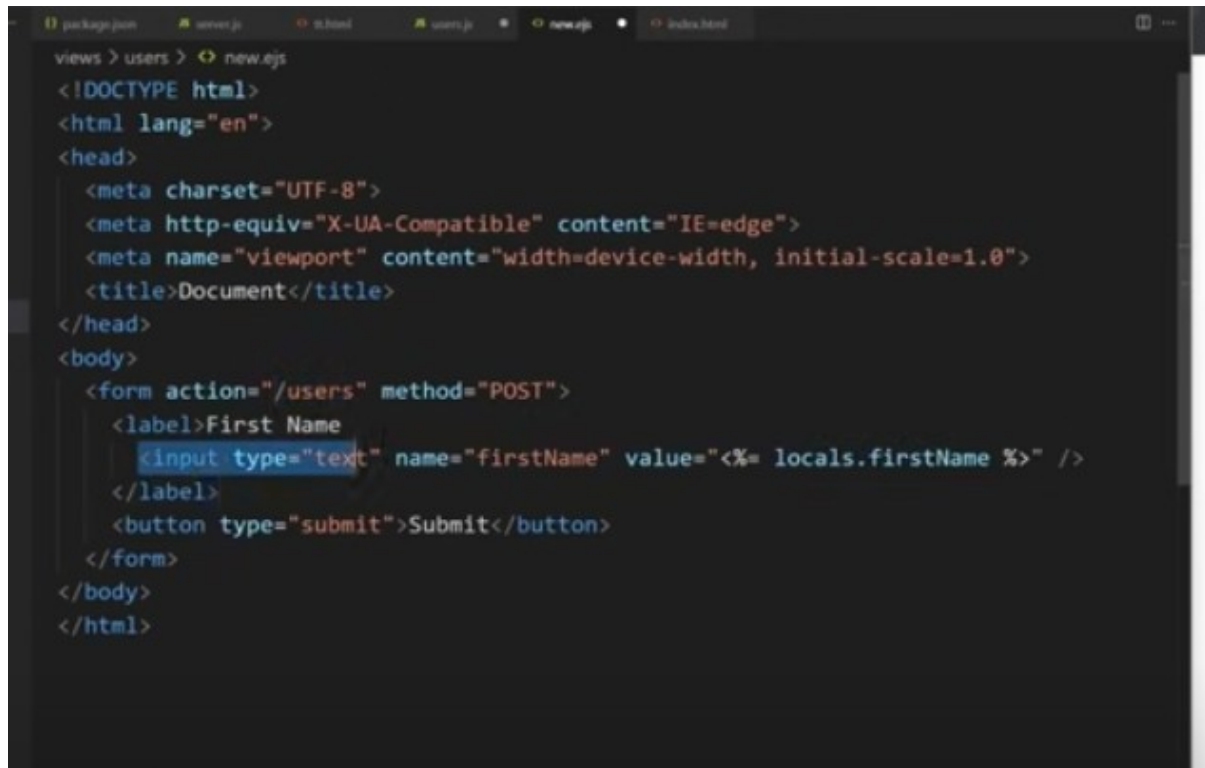
const users = [{ name: "Kyle" }, { name: "Sally" }]
router.param("id", (req, res, next, id) => {
  req.user = users[id]
  next()
})

```

Form input to save as json

setup

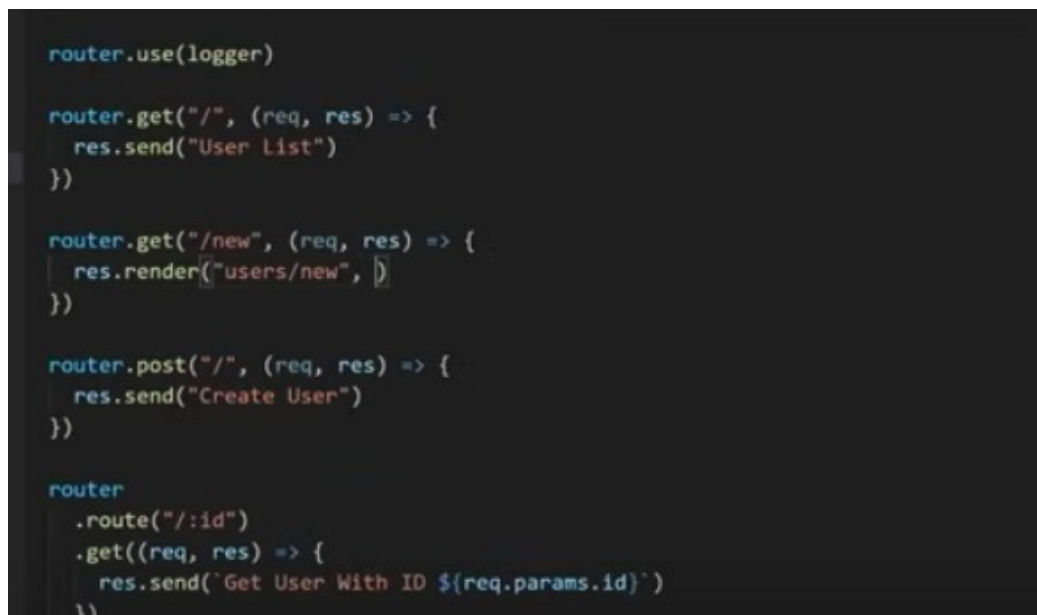
1. form.ejs file create in VIEWS

A screenshot of a code editor with a dark theme. The file explorer on the left shows a project structure with 'package.json', 'server.js', 'n.html', 'users.js', 'new.ejs', and 'index.html'. The main editor area displays the content of 'new.ejs'. The code is an HTML form with a text input field for 'First Name' and a 'Submit' button. The input field's value is set to a local variable 'locals.firstName'.

```
views > users > new.ejs
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <form action="/users" method="POST">
    <label>First Name
      <input type="text" name="firstName" value="<%= locals.firstName %>" />
    </label>
    <button type="submit">Submit</button>
  </form>
</body>
</html>
```

the form has an action to our router, post "users/" AND method is POST

2. the route /new has a render function to render the form.ejs

A screenshot of a code editor with a dark theme showing Express.js router configuration. The code defines routes for GET, POST, and a parameterized GET request.

```
router.use(logger)

router.get("/", (req, res) => {
  res.send("User List")
})

router.get("/new", (req, res) => {
  res.render("users/new", {})
})

router.post("/", (req, res) => {
  res.send("Create User")
})

router
  .route("/:id")
  .get((req, res) => {
    res.send(`Get User With ID ${req.params.id}`)
  })
```

3. we want not have a render CREATE USER – we will have the value of the form input ...
we want to have the value of the input field

```
router.get("/new", (req, res) => {  
  res.render("users/new")  
})  
  
router.post("/", (req, res) => {  
  req.body.firstName  
})  
  
router  
  .route("/:id")
```

4. type in the name and press the button
an error will appear...we need an middleware calles express.urlencoded({extended:true})

```
const express = require("express")  
const app = express()  
  
app.use(express.static("public"))  
app.use(express.urlencoded({ extended: true })))  
  
app.set("view engine", "ejs")  
  
const userRouter = require("./routes/users")  
  
app.use("/users", userRouter)
```

EXPRESS JS

EXAMPLE with EJS, input field and public styling

1. install ejs and express to node application
2. create folder

```
express-ejs-app
├── views
│   ├── index.ejs
│   └── users.ejs
├── public
│   └── style.css
├── server.js
└── package.json
```

views/: Contains the EJS templates.

public/: Contains static files (like CSS).

server.js: Contains the Express.js server logic.

3. setup server and routes

```
import express from 'express';
import path from 'path';
import { fileURLToPath } from 'url';

const app = express();
const port = 3000;

// Configure EJS as the template engine
app.set('view engine', 'ejs');

// Serve static files (e.g., CSS) from the "public" folder
const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
app.use(express.static(path.join(__dirname, 'public')));

// Middleware to parse incoming POST data
app.use(express.urlencoded({ extended: true }));

// Sample data for users
let users = [
  { name: 'Alice', age: 25 },
  { name: 'Bob', age: 30 },
];

// ROUTES
// Root route - renders the index.ejs view
app.get('/', (req, res) => {
  res.render('index', { title: 'Home Page' });
});

// GET /users - renders a form and the list of users
app.get('/users', (req, res) => {
  res.render('users', { users });
});

// POST /users - adds a new user and redirects to /users
app.post('/users', (req, res) => {
  const { name, age } = req.body;
  if (name && age) {
    users.push({ name, age: parseInt(age) });
  }
  res.redirect('/users');
});

// Start the server
app.listen(port, () => {
  console.log(`Server is running on http://localhost:${port}`);
});
```



4. create views

HINT: /views path is required in ejs.

/views/index.ejs

```
<!-- views/index.ejs -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title><%= title %></title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>Welcome to the Home Page</h1>
  <p><a href="/users">Go to Users</a></p>
</body>
</html>
<!--end index.ejs-->
```

/views/users.ejs

```
<!-- views/users.ejs -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Users</title>
  <link rel="stylesheet" href="/style.css">
</head>
<body>
  <h1>User List</h1>

  <!-- User List -->
  <ul>
    <% users.forEach(user => { %>
      <li><%= user.name %> - <%= user.age %> years old</li>
    <% }); %>
  </ul>

  <!-- User Form -->
  <h2>Add a New User</h2>
  <form action="/users" method="POST">
    <label for="name">Name:</label>
    <input type="text" id="name" name="name" required>

    <label for="age">Age:</label>
    <input type="number" id="age" name="age" required>

    <button type="submit">Add User</button>
  </form>

  <p><a href="/">Go Back to Home</a></p>
</body>
</html><!--end user.ejs-->
```

5. add styling to the project

```
/* public/style.css */
body {
  font-family: Arial, sans-serif;
  margin: 20px;
}

h1, h2 {
  color: #333;
}

ul {
  list-style-type: none;
  padding: 0;
}

ul li {
  background-color: #f9f9f9;
  margin: 5px 0;
  padding: 10px;
  border-radius: 4px;
}

form {
  margin-top: 20px;
}

form label {
  display: block;
  margin-top: 10px;
}

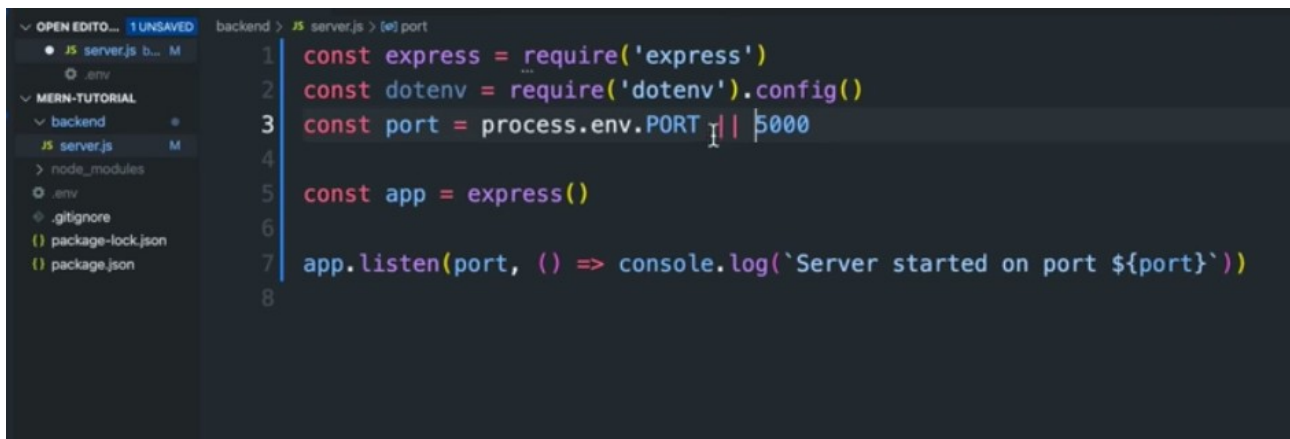
form input {
  padding: 5px;
  margin-top: 5px;
  width: 200px;
}
```

Project with Express

Create Rest API with NodeJS and MongoDB

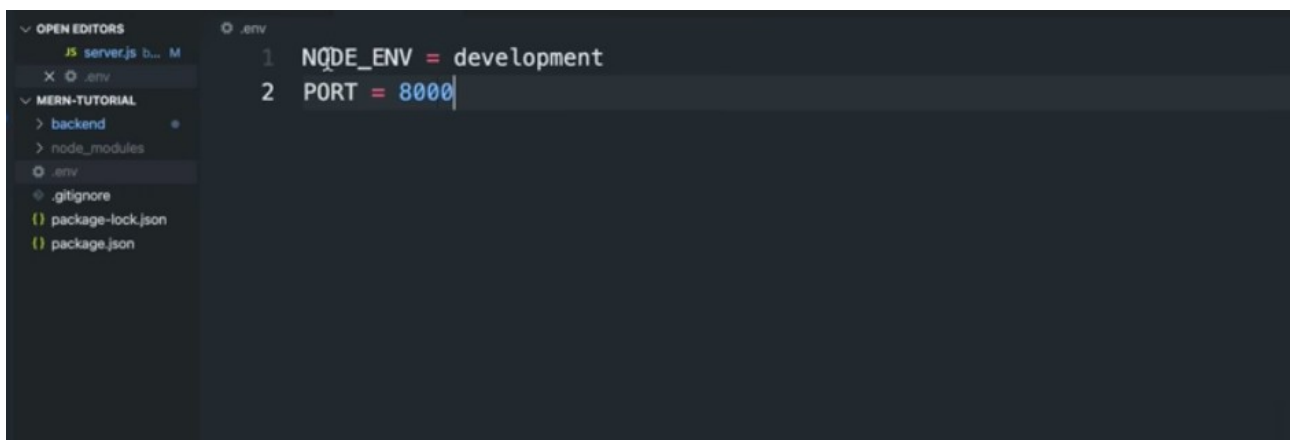
<https://youtu.be/-0exw-9YJBo?t=381>

set up node server

A screenshot of a code editor showing a file named 'server.js' in the 'backend' directory. The code is written in JavaScript and uses Express.js. It imports 'express' and 'dotenv', sets the port from 'process.env.PORT' or defaults to 5000, creates an Express app, and listens on that port with a log message.

```
1 const express = require('express')
2 const dotenv = require('dotenv').config()
3 const port = process.env.PORT || 5000
4
5 const app = express()
6
7 app.listen(port, () => console.log(`Server started on port ${port}`))
8
```

server port info comes from an .env file

A screenshot of a code editor showing a file named '.env'. It contains two lines of environment variables: 'NODE_ENV = development' and 'PORT = 8000'. The 'PORT' line is currently selected.

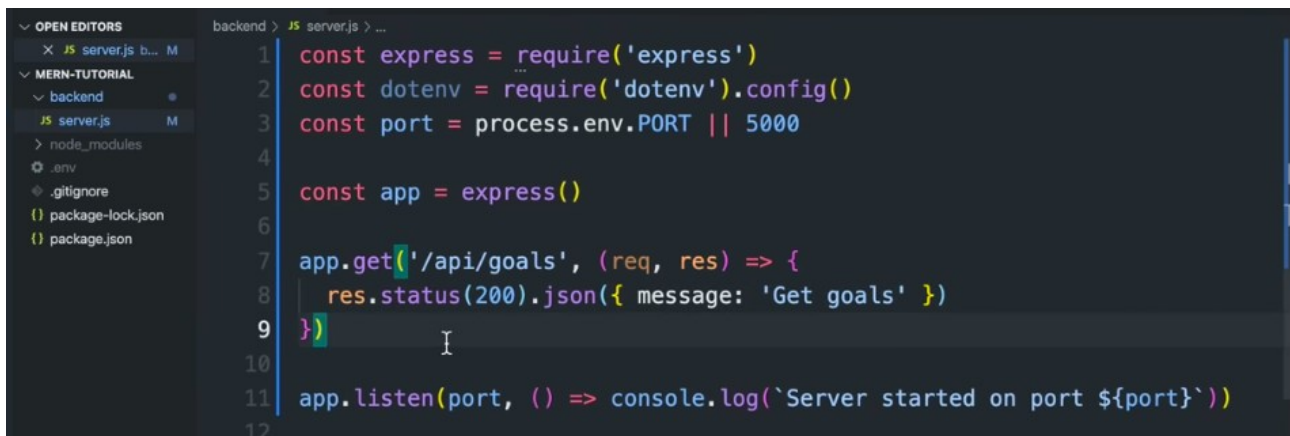
```
1 NODE_ENV = development
2 PORT = 8000
```

to check APIs you can install POSTMAN

Setup API Route

Route return two parameter...

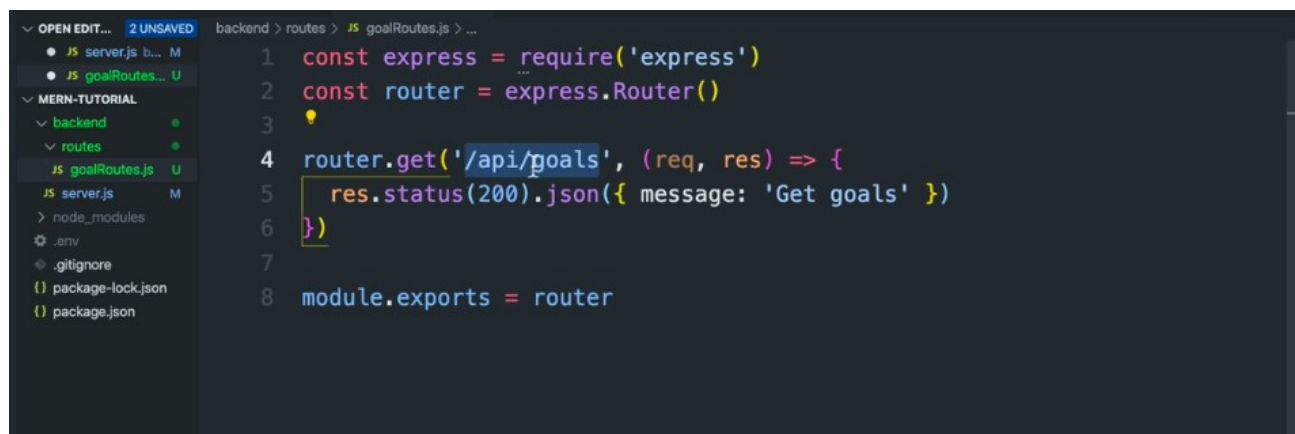
1. statusrou
2. json



```
1 const express = require('express')
2 const dotenv = require('dotenv').config()
3 const port = process.env.PORT || 5000
4
5 const app = express()
6
7 app.get('/api/goals', (req, res) => {
8   res.status(200).json({ message: 'Get goals' })
9 })
10
11 app.listen(port, () => console.log(`Server started on port ${port}`))
12
```

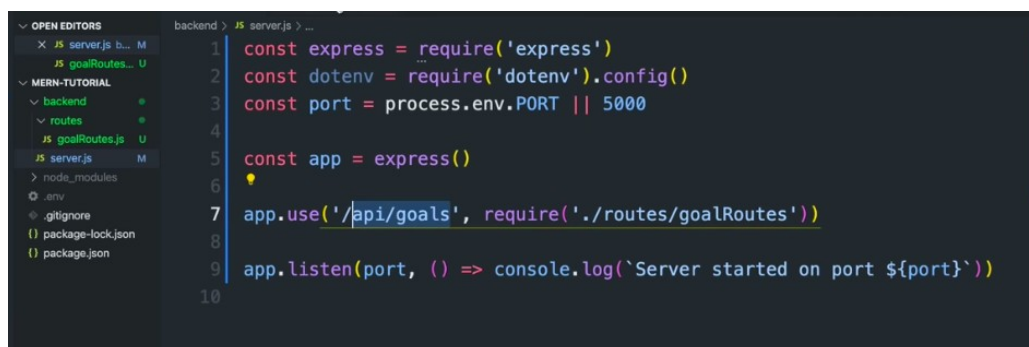
Create Router for better structuring of the above API

1. Create route/goalRoutes.js file (naming is up to you)
2. in the goalRoutes.js you put the following



```
1 const express = require('express')
2 const router = express.Router()
3
4 router.get('/api/goals', (req, res) => {
5   res.status(200).json({ message: 'Get goals' })
6 })
7
8 module.exports = router
```

initialize the routes file in the index.js with app.use()



```
1 const express = require('express')
2 const dotenv = require('dotenv').config()
3 const port = process.env.PORT || 5000
4
5 const app = express()
6
7 app.use('/api/goals', require('./routes/goalRoutes'))
8
9 app.listen(port, () => console.log(`Server started on port ${port}`))
10
```

Add more Route options to the goalRoutes.js

```
1  const express = require('express')
2  const router = express.Router()
3
4  router.get('/', (req, res) => {
5    res.status(200).json({ message: 'Get goals' })
6  })
7
8  router.post('/', (req, res) => {
9    res.status(200).json({ message: 'Set goal' })
10 })
11
12 router.put('/:id', (req, res) => {
13   res.status(200).json({ message: `Update goal ${req.params.id}` })
14 })
15
16 router.delete('/:id', (req, res) => {
17   res.status(200).json({ message: `Delete goal ${req.params.id}` })
18 })
```

.post() will set a new goal

.put() will update an existing goal – don't forget the param which points to the goal you want to update

.delete() will delete the goal – param has to be included

Create CONTROLLER to manage node project proper

instead of writing all functionality in the index.js file it is nicer to decouple the function of the routes in an goalController.js

the example below is only the GET request ..

check the comments above and use this to explain the core functionality of the function.

```
1 // @desc    Get goals
2 // @route    GET /api/goals
3 // @access   Private
4 const getGoals = (req, res) => {
5   res.status(200).json({ message: 'Get goals' })
6 }
7
8 module.exports = {
9   getGoals,
10 }
```

and import the controller file to index.js – LINE 5

```
1 const express = require('express')
2 const router = express.Router()
3 const { getGoals } = require('../controllers/goalController')
4
```

merge route actions which have the same route

.get and .post has the “/” route.

.put and .delete has the “/:id” route.

```
6   updateGoal,
7   deleteGoal,
8 } = require('../controllers/goalController')
9
10 router.route('/').get(getGoals).post(setGoal)
11
12 router.put('/:id', updateGoal)
13
14 router.delete('/:id', deleteGoal)
15
16 module.exports = router
```

Return body data in post request

Getting body data is only possible with a middleware function to accepts body data encoded.

We will add 2 middleware with .use() method

1. .json formatter
2. encoded



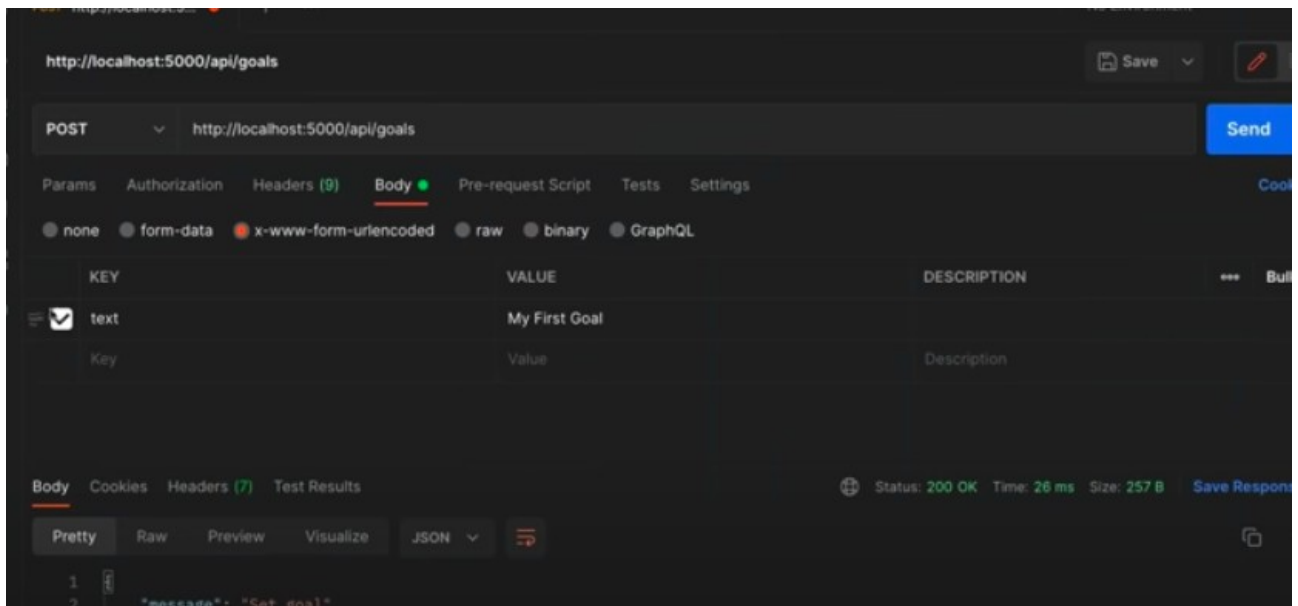
```
1 const express = require('express')
2 const dotenv = require('dotenv').config()
3 const port = process.env.PORT || 5000
4
5 const app = express()
6
7 app.use(express.json())
8 app.use(express.urlencoded({ extended: false }))
9
10 app.use('/api/goals', require('./routes/goalRoutes'))
11
12 app.listen(port, () => console.log(`Server started on port ${port}`))
13
```

urlencoded(options?: bodyParser.OptionsUrlencoded): createServer.NextHandleFunction

Returns middleware that only parses urlencoded bodies and only looks at requests where the Content-Type header matches the type option

After we included the middleware, we can get the body.data from the api.

We used postman to post data as text (see set up below)



After setting up the postman request, we will check the response.

If no response incoming, we will use the express error handle function (default) to inform the user.

```

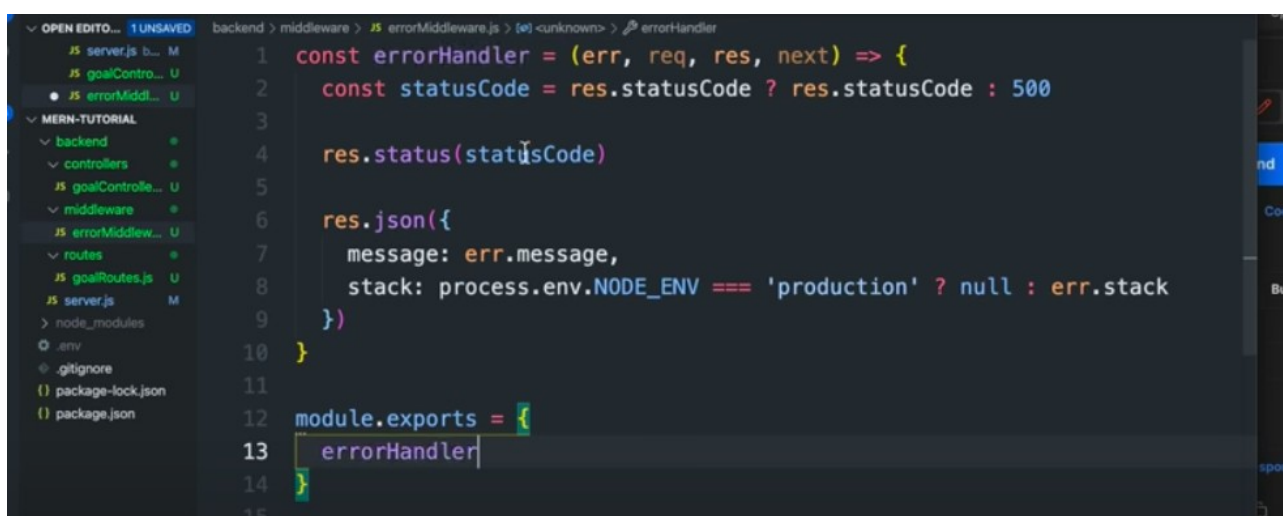
8 // @desc    Set goal
9 // @route    POST /api/goals
10 // @access   Private
11 const setGoal = (req, res) => {
12   if (!req.body.text) {
13     res.status(400)
14     throw new Error('Please add a text field')
15   }
16
17   res.status(200).json({ message: 'Set goal' })
18 }
19

```

Customize the error message which will come from `express.error()`

To manage the error handling as a custom middleware solution, we have to create a folder in the main project folder with name **middleware**.

The file can have the name **errorMiddleware.js**

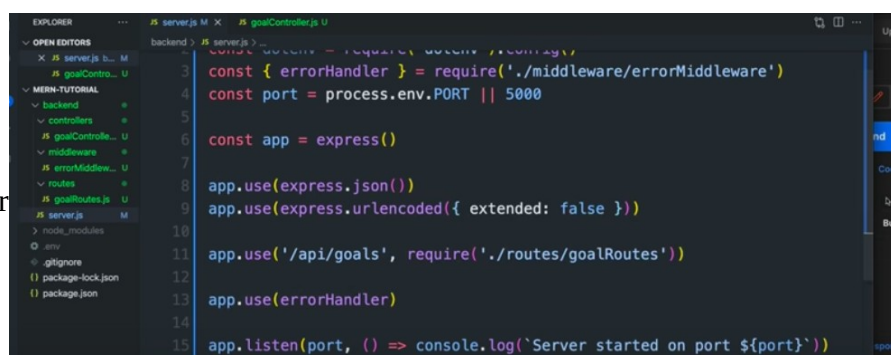


```

1 const errorHandler = (err, req, res, next) => {
2   const statusCode = res.statusCode ? res.statusCode : 500
3
4   res.status(statusCode)
5
6   res.json({
7     message: err.message,
8     stack: process.env.NODE_ENV === 'production' ? null : err.stack
9   })
10 }
11
12 module.exports = {
13   errorHandler
14 }
15

```

after setting up the error handler, we have to include (line 3) the middleware to the index.js in line 13 we included with `app.use` the middleware – after the router



```

1 const { errorHandler } = require('./middleware/errorMiddleware')
2 const port = process.env.PORT || 5000
3
4 const app = express()
5
6 app.use(express.json())
7 app.use(express.urlencoded({ extended: false }))
8
9 app.use('/api/goals', require('./routes/goalRoutes'))
10
11 app.use(errorHandler)
12
13 app.listen(port, () => console.log(`Server started on port ${port}`))
14

```

Controller update to async handling

Before we include the database, we have to tweak the goalController functions to become an async/await function. This is because with mongoose we get an promise.

We do update all controller functions to async...

update all functions like the example below.

```
23 const updateGoal = async (req, res) => {
24   res.status(200).json({ message: `Update goal ${req.params.id}` })
25 }
```

Install express async handler dependency.

NPM in express-async-handler

after installing we have to import the module to the goalController.js at the top.
Each function has to be wrapped with the method asyncHandler.

```
1 const asyncHandler = require('express-async-handler')
2 // @access private
6 const getGoals = asyncHandler(async (req, res) => {
7   res.status(200).json({ message: 'Get goals' })
8 })
```